

1.1-1.3.

Для нахождения общего решения СЛАУ расширенная матрица системы приводится к приведенному ступенчатому виду методом Гаусса-Жордана. Столбцы с ведущими единицами соответствуют базисным переменным, которые затем линейно выражаются через свободные. Если система недоопределена (неизвестных больше, чем уравнений), можно аналитически исключить часть переменных путем алгебраических преобразований, чтобы найти прямую линейную зависимость между оставшимися переменными (например, выразить y через x).

```
In [1]: # Задание 1.1
import numpy as np
import scipy.linalg as la
import sympy as sp
import matplotlib.pyplot as plt

sp.init_printing()
A=np.array([[1,5,-2,0],[-3,1,9,-5],[4,-8,-1,7]])
b=np.array([-7,9,0]).reshape(3,1)
A_b=np.hstack([A,b])
sp.Matrix(A_b).rref()
```

```
Out[1]:
```

$$\left(\begin{bmatrix} 1 & 0 & 0 & \frac{8}{7} & -\frac{11}{7} \\ 0 & 1 & 0 & -\frac{2}{7} & -\frac{6}{7} \\ 0 & 0 & 1 & -\frac{1}{7} & \frac{4}{7} \end{bmatrix}, (0, 1, 2) \right)$$

```
In [2]: # Задание 1.2
A2 = sp.Matrix([
    [5, -1, 5, 9, -2],
    [3, 0, 6, 9, -1],
    [2, 0, 4, 6, -1]
])
b2 = sp.Matrix([-3, 4, 0])

Ab2 = A2.row_join(b2)
print("a) rref(A|b) =")
display(Ab2.rref()[0])

vars2 = sp.symbols('x1:6')
sol_2 = sp.linsolve((A2, b2), vars2)

print("6) x =")
display(sol_2)
```

a) $\text{rref}(A|b) =$

$$\begin{bmatrix} 1 & 0 & 2 & 3 & 0 & 4 \\ 0 & 1 & 5 & 6 & 0 & 7 \\ 0 & 0 & 0 & 0 & 1 & 8 \end{bmatrix}$$

6) $x =$

$$\{(-2x_3 - 3x_4 + 4, -5x_3 - 6x_4 + 7, x_3, x_4, 8)\}$$

```
In [3]: # Задание 1.3
x, y, z, u = sp.symbols('x y z u')
eqs = [
    32*x + 97*y + 90*z + 48*u - 47,
    35*x + 84*y + 91*z + 54*u - 40,
    17*x + 18*y + 21*z + 13*u - 37
]
sol = sp.solve(eqs, (y, z, u))
y_expr = sp.simplify(sol[y])
a_const = y_expr.coeff(x, 0)
b_const = y_expr.coeff(x, 1)
print(f"y(x) = {y_expr}")
print(f"a = {a_const}, b = {b_const}")
```

```
y(x) = 14027 - 4262*x
a = 14027, b = -4262
```

1.4.

Умножение матриц не обладает свойством коммутативности и выполняется по правилу умножения строки на столбец. Сложение матриц выполняется поэлементно и применимо только к матрицам одинаковой размерности.

```
In [4]: # Задание 1.4
M1 = np.array([[3, 0, 2], [0, 1, 3], [2, 2, 0], [0, 1, 0]])
M2 = np.array([[1, 2, -1, 2], [-2, 1, 1, 2], [2, 1, 1, 2]])
M3 = np.array([[0, -4, 6, 1], [2, 2, -5, -2], [2, -2, 6, 4], [1, 3, 0, 1]])
res = M1 @ M2 + M3
res
```

```
Out[4]: array([[ 7,  4,  5, 11],
               [ 6,  6, -1,  6],
               [ 0,  4,  6, 12],
               [-1,  4,  1,  3]])
```

1.5 и 1.6.

Для матриц с заданными правилами формирования элементов (минимум, максимум или модуль разности индексов) определители вычисляются путем приведения к треугольному виду. Согласно теореме Адамара, определитель матрицы ограничен длинами ее векторов. Максимальный определитель достигается, когда матрица состоит только из элементов 1 и -1 (нулевые элементы не увеличивают значение). Поиск оптимума для небольших матриц эффективно выполняется методом Монте-Карло.

```
In [5]: # Задание 1.5
n_test = 5
A_a = sp.Matrix(n_test, n_test, lambda i, j: min(i + 1, j + 1))
A_b = sp.Matrix(n_test, n_test, lambda i, j: max(i + 1, j + 1))
A_v = sp.Matrix(n_test, n_test, lambda i, j: abs((i + 1) - (j + 1)))
```

```
print("a) det =", A_a.det())
print("б) det =", A_b.det())
print("в) det =", A_v.det())
```

a) det = 1
 б) det = 5
 в) det = 32

```
In [6]: # Задание 1.6
max_det = 0
best_M = None

for _ in range(100000):
    M = np.random.choice([-1, 1], size=(6, 6))
    d = abs(np.linalg.det(M))
    if d > max_det:
        max_det = round(d)
        best_M = M
        if max_det == 160:
            break

print(f"Максимальный определитель: {max_det}")
display(sp.Matrix(best_M))
```

Максимальный определитель: 160

$$\begin{bmatrix} -1 & 1 & 1 & 1 & -1 & 1 \\ 1 & -1 & 1 & 1 & 1 & 1 \\ 1 & -1 & -1 & 1 & -1 & -1 \\ 1 & 1 & -1 & -1 & -1 & 1 \\ 1 & -1 & 1 & -1 & -1 & -1 \\ 1 & 1 & 1 & 1 & 1 & -1 \end{bmatrix}$$

1.7 и 1.8.

Прямое вычисление определителя или обратной матрицы для больших размерностей приводит к арифметическому переполнению или нехватке памяти. Определитель вычисляется как произведение диагональных элементов (через сумму логарифмов для обхода переполнения), а обратная матрица для треугольной остается треугольной, где элементы рассчитываются по точным формулам.

```
In [7]: # Задание 1.7
n7 = 80000
d = np.linspace(0.99, 1.01, n7)
log_diag_tail = np.sum(np.log(d[2:]))
det_top2 = d[0] * d[1] - 0.57 * 0.03
det_total = np.exp(np.log(det_top2) + log_diag_tail)
inv_top2_trace = (d[0] + d[1]) / det_top2
trace_inv = inv_top2_trace + np.sum(1.0 / d[2:])
print(f"1) det = {det_total:.3f}")
print(f"2) trace = {trace_inv:.3f}")
```

1) det = 0.259
 2) trace = 80002.703

```
In [8]: # Задание 1.8
n8 = 43000
q = (1.07)**(1.0 / (n8 - 1))
d = 1.0 * (q ** np.arange(n8))
r = np.linspace(1.0, 1.10, n8)
b_diag = 1.0 / d
b_row0_rest = - r[1:] / (d[0] * d[1:])
min_elem = min(b_diag.min(), b_row0_rest.min())
sum_all = b_diag.sum() + b_row0_rest.sum()
print(f"1) min = {min_elem:.3f}")
print(f"2) sum = {sum_all:.1f}")
```

1) min = -1.028
2) sum = -2054.4

1.9.

Матричный ряд, представляющий собой геометрическую прогрессию, сходится при условии, что спектральный радиус матрицы меньше единицы. Предел бесконечной суммы такого ряда равен обратной матрице $(I - A)^{-1}$, умноженной на начальный элемент прогрессии.

```
In [9]: # Задание 1.9
gamma = 0.057
Amat = np.array([[7 * gamma, 14 * gamma], [9 * gamma, 1 * gamma]])
Imat = np.eye(2)
S_4_70 = sum(np.linalg.matrix_power(Amat, i) for i in range(4, 71))
print("1) max(S_4,70) =", S_4_70.max())
inv_IA = np.linalg.inv(Imat - Amat)
B_4 = inv_IA @ np.linalg.matrix_power(Amat, 4)
print("2) max(B_4) =", B_4.max())
for k_val in range(1, 200):
    Bk = inv_IA @ np.linalg.matrix_power(Amat, k_val)
    if Bk.max() < 0.0002:
        print("3) k =", k_val)
        break
```

1) max(S_4,70) = 3.609679030803459
2) max(B_4) = 3.611174678109661
3) k = 89

1.10.

Определитель матрицы с символьной переменной раскладывается по строкам или столбцам согласно теореме Лапласа. В результате получается алгебраический полином, из которого можно аналитически выделить числовые коэффициенты при соответствующих степенях неизвестной переменной.

```
In [10]: # Задание 1.10
x_sym = sp.Symbol('x')
Mmat = sp.Matrix([
    [7, 5, 5, x_sym],
```

```

    [2, 2, 6, 9],
    [3, x_sym, 8, 0],
    [3, 1, 1, 0]
])
poly_det = sp.expand(Mmat.det())
a_coeff = poly_det.coeff(x_sym, 2)
b_coeff = poly_det.coeff(x_sym, 1)
print("1) a =", a_coeff)
print("2) b =", b_coeff)

```

1) a = 16
2) b = -116

1.11 и 1.12.

Согласно теореме Люка, нечетность биномиальных коэффициентов проверяется побитовой операцией И, что при отрисовке матрицы формирует фрактал (треугольник Серпинского). Матрицы Сильвестра-Адамара строятся рекурсивно с использованием тензорного произведения. Для графического отображения в виде функций Уолша их строки дополнительно сортируются по количеству смен знака.

```

In [11]: # Задание 1.11
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

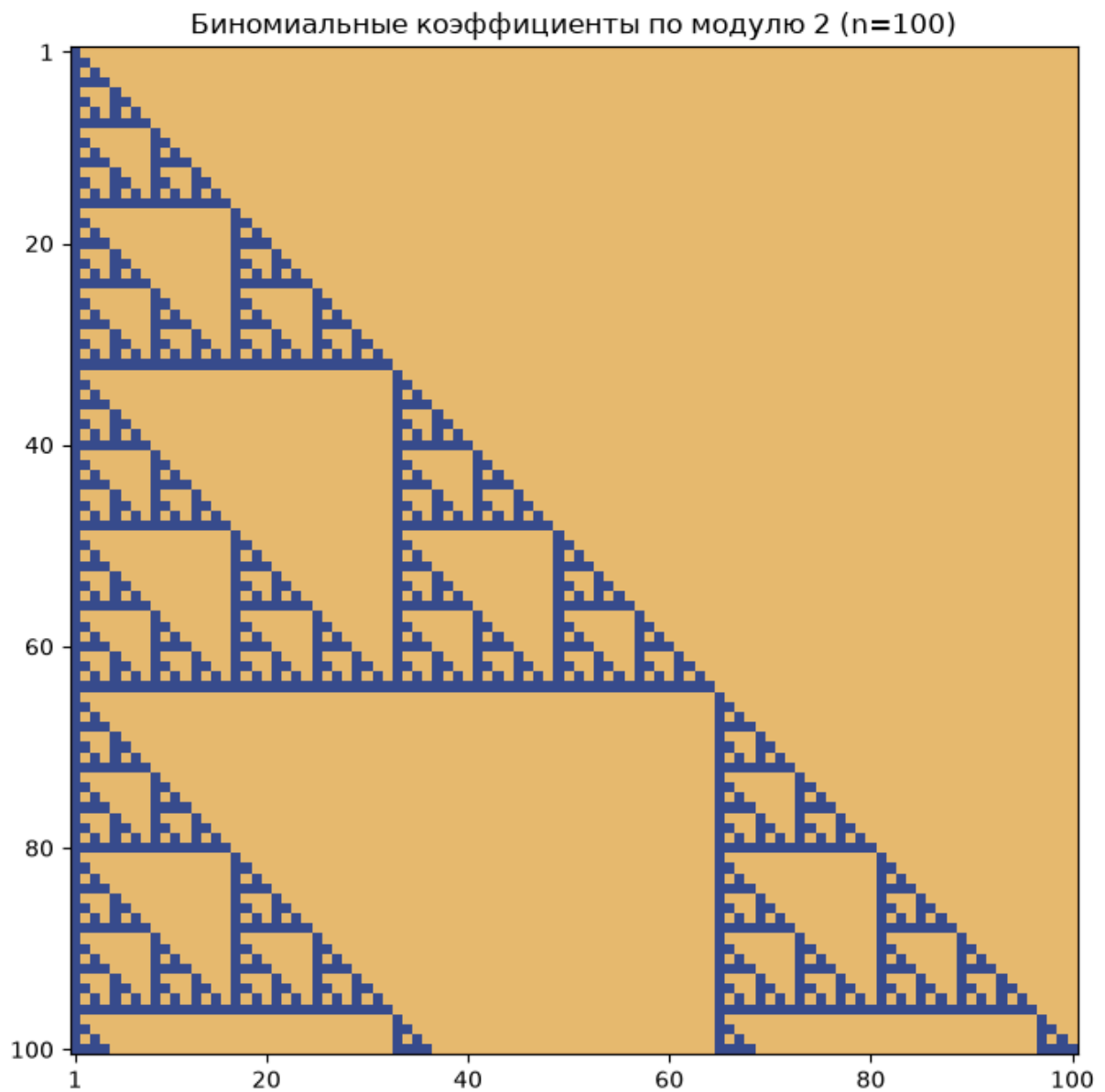
n = 100
grid = np.zeros((n, n), dtype=int)

for i in range(n):
    for j in range(i + 1):
        if (j & i) == j:
            grid[i, j] = 1

cmap = ListedColormap(['#e9b96e', '#3b4d8c'])

plt.figure(figsize=(8, 8))
plt.imshow(grid, cmap=cmap)
plt.title(f"Биномиальные коэффициенты по модулю 2 (n={n})")
plt.xticks([0, 19, 39, 59, 79, 99], [1, 20, 40, 60, 80, 100])
plt.yticks([0, 19, 39, 59, 79, 99], [1, 20, 40, 60, 80, 100])
plt.show()

```



```
In [12]: # Задание 1.12
from sympy.physics.quantum import TensorProduct

H1 = sp.Matrix([[1]])
H2 = sp.Matrix([
    [1, 1],
    [1, -1]
])
H4_explicit = sp.Matrix([
    [1, 1, 1, 1],
    [1, -1, 1, -1],
    [1, 1, -1, -1],
    [1, -1, -1, 1]
])

print("a) H1 в явном виде:")
display(H1)
print("H2 в явном виде:")
display(H2)
```

```

print("H4 в явном виде:")
display(H4_explicit)

H4_tensor = TensorProduct(H2, H2)

print("6) Результат вычисления H2  $\otimes$  H2:")
display(H4_tensor)

def get_H(k):
    if k == 0:
        return H1
    if k == 1:
        return H2
    return TensorProduct(H2, get_H(k-1))

H8 = get_H(3)
print("в) Рекурсивное построение для k=3 (H8 = H2  $\otimes$  H4):")
display(H8)

def hadamard_walsh(k):
    H = np.array([[1]])
    for _ in range(k):
        H = np.block([[H, H], [H, -H]])

    sign_changes = np.sum(np.abs(np.diff(H, axis=1)) > 0, axis=1)

    return H[np.argsort(sign_changes)]

H64 = hadamard_walsh(6)

cmap = ListedColormap(['#3b4d8c', '#e66101'])

plt.figure(figsize=(8, 8))
plt.imshow(H64, cmap=cmap)
plt.title("Матрица Адамара")
plt.xticks([0, 19, 39, 63], [1, 20, 40, 64])
plt.yticks([0, 19, 39, 63], [1, 20, 40, 64])
plt.show()

```

а) H1 в явном виде:

```
[1]
```

H2 в явном виде:

$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

H4 в явном виде:

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

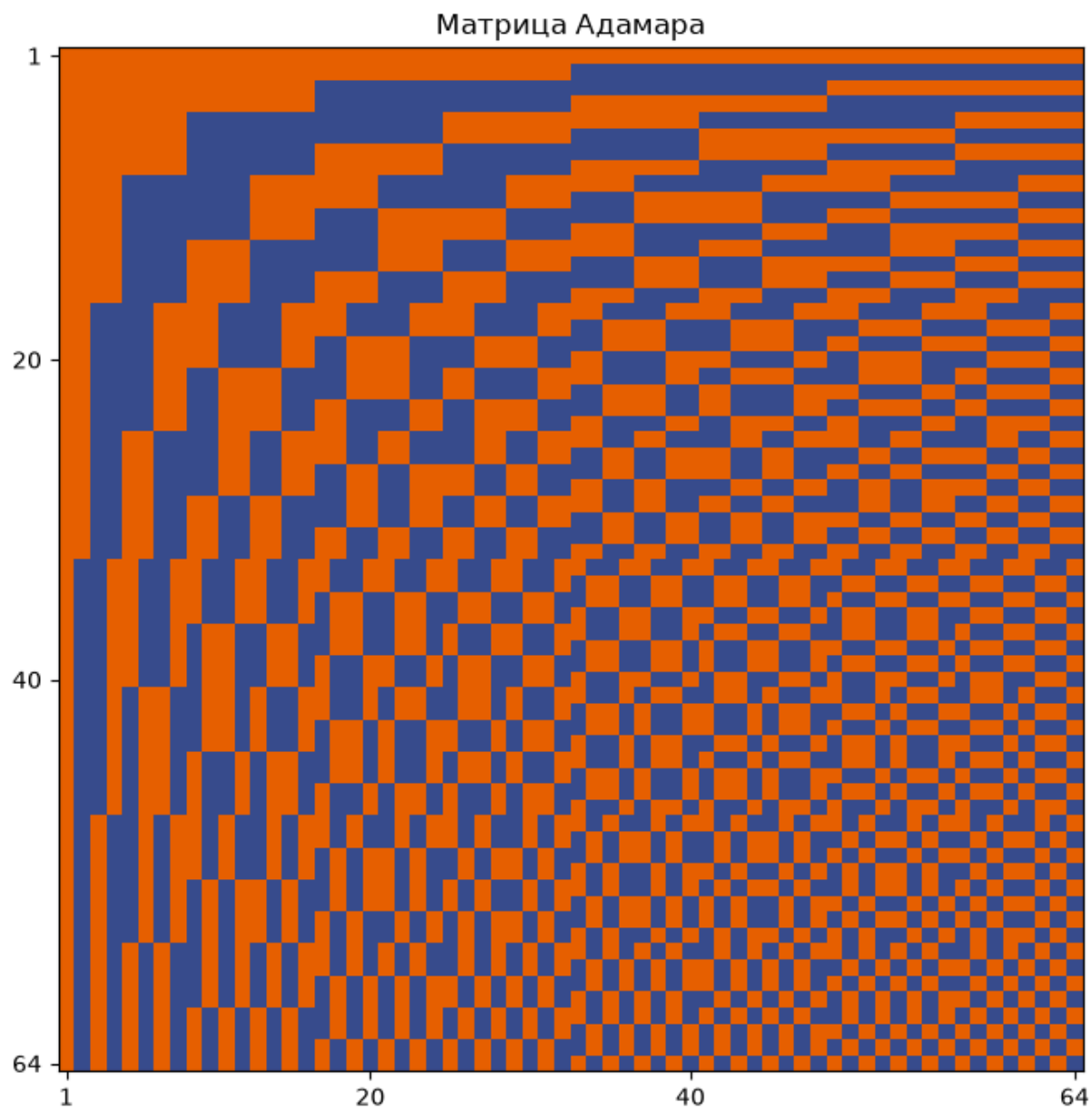
б) Результат вычисления H2 $\otimes$ H2:



$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

в) Рекурсивное построение для $k=3$ ($H_8 = H_2 \otimes H_4$):

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 \end{bmatrix}$$



1.13 и 1.14.

Собственные значения являются корнями характеристического уравнения матрицы. Минимальный многочлен аннулирует исходную матрицу и имеет наименьшую возможную степень среди всех таких многочленов, при этом его корнями выступают все собственные значения без учета алгебраической кратности. Он находится путем перебора комбинаций степеней множителей характеристического многочлена. Для каждого собственного значения можно найти и нормировать соответствующий собственный вектор.

```
In [13]: # Задание 1.13
A13 = sp.Matrix([[14, 10, 21], [12, 15, 26], [18, 11, 27]])
evals_evecs = A13.eigenvecs()
all_pairs = []
for val, mult, vecs in evals_evecs:
    val_c = complex(val.evalf())
    vec_c = np.array(vecs[0].evalf(), dtype=complex).flatten()
    all_pairs.append((val_c, vec_c))
all_pairs.sort(key=lambda item: item[0].imag, reverse=True)
best_val, best_vec = all_pairs[0]
X_norm = best_vec / best_vec[0]
print(f"1) Im(Z) = {best_val.imag:.6f}")
print(f"2) Re(U) = {X_norm[1].real:.6f}")
```

```
1) Im(Z) = 0.344789
2) Re(U) = 3.747820
```

```
In [14]: # Задание 1.14
import itertools
import sympy as sp

A14 = sp.Matrix([
    [-3, 7, -6, -1, 8, -3, -2],
    [1, -5, 8, -3, -5, 4, -5],
    [1, 3, -4, 6, -2, 0, 8],
    [-1, 11, -12, 9, 3, -3, 9],
    [-3, 11, -12, 5, 8, -4, 6],
    [-3, 2, 1, -4, 4, 1, -7],
    [1, -5, 6, -2, -4, 3, -2]
])

lam_sym = sp.Symbol('lambda')

char_poly = A14.charpoly(lam_sym).as_expr()
factors = sp.factor_list(char_poly)[1]

powers = [range(1, m + 1) for f, m in factors]
combinations = list(itertools.product(*powers))
combinations.sort(key=sum)

min_poly = None

for p in combinations:
    test_poly = 1
    for i, (f, m) in enumerate(factors):
        test_poly *= f**p[i]
```

```

expanded = sp.expand(test_poly)
coeffs = sp.Poly(expanded, lam_sym).all_coeffs()

degree = len(coeffs) - 1
M_val = sp.zeros(7, 7)
for i, c in enumerate(coeffs):
    M_val += c * (A14 ** (degree - i))

if M_val == sp.zeros(7, 7):
    min_poly = test_poly
    break

min_poly

```

Out[14]: $\lambda^3(\lambda - 1)^2$

1.15-1.17.

По спектральной теореме любая симметричная вещественная матрица имеет ортогональный базис из собственных векторов, который приводится к ортонормированному виду процессом Грама-Шмидта. Это позволяет ортогональным преобразованием координат привести квадратичную форму к главным осям (сумме квадратов). Аналогичный алгоритм используется для классификации кривых второго порядка: матрица перехода задает поворот системы координат, а собственные значения становятся коэффициентами канонического уравнения (совпадение их знаков указывает на эллипс, а разные знаки на гиперболу).

In [15]: *# Задание 1.15*

```

A15 = sp.Matrix([[17, -8, 4], [-8, 17, -4], [4, -4, 11]])

ortho_vecs = []
evals = []
for val, mult, vecs in A15.eigenvects():
    ortho_vecs.extend(sp.GramSchmidt(vecs, orthonormal=True))
    evals.extend([val] * mult)

P_mat = sp.Matrix.hstack(*ortho_vecs)
D_mat = sp.diag(*evals)

display(D_mat)
display(P_mat)

```

$$\begin{bmatrix} 9 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 27 \end{bmatrix}$$

$$\begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{6} & \frac{2}{3} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{6} & -\frac{2}{3} \\ 0 & \frac{2\sqrt{2}}{3} & \frac{1}{3} \end{bmatrix}$$

```
In [16]: # Задание 1.16
A_b1 = sp.Matrix([[3, 4], [4, -3]])
A_b2 = sp.Matrix([[4, -2], [-2, 1]])

def get_ortho_PD(A):
    ortho_vecs = []
    evals = []
    for val, mult, vecs in A.eigenvects():
        ortho_vecs.extend(sp.GramSchmidt(vecs, orthonormal=True))
        evals.extend([val] * mult)
    return sp.Matrix.hstack(*ortho_vecs), sp.diag(*evals)

P1, D1 = get_ortho_PD(A_b1)
P2, D2 = get_ortho_PD(A_b2)

Q_ortho = sp.diag(P1, P2)
D_diag = sp.diag(D1, D2)

display(D_diag)
display(Q_ortho)
```

$$\begin{bmatrix} -5 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 \end{bmatrix}$$

$$\begin{bmatrix} -\frac{\sqrt{5}}{5} & \frac{2\sqrt{5}}{5} & 0 & 0 \\ \frac{2\sqrt{5}}{5} & \frac{\sqrt{5}}{5} & 0 & 0 \\ 0 & 0 & \frac{\sqrt{5}}{5} & -\frac{2\sqrt{5}}{5} \\ 0 & 0 & \frac{2\sqrt{5}}{5} & \frac{\sqrt{5}}{5} \end{bmatrix}$$

```
In [17]: # Задание 1.17
A17a = sp.Matrix([[5, 2], [2, 8]])

ortho_vecs = []
evals = []
for val, mult, vecs in A17a.eigenvects():
    ortho_vecs.extend(sp.GramSchmidt(vecs, orthonormal=True))
    evals.extend([val] * mult)

P17a = sp.Matrix.hstack(*ortho_vecs)
D17a = sp.diag(*evals)

print("a) Эллипс, каноническое уравнение: X^2 / 4 + Y^2 / 9 = 1")
print("b) Гипербола, каноническое уравнение: X^2 / 4 - Y^2 / 9 = 1")
```

a) Эллипс, каноническое уравнение: $X^2 / 4 + Y^2 / 9 = 1$

b) Гипербола, каноническое уравнение: $X^2 / 4 - Y^2 / 9 = 1$

1.18-1.20.

Знакоопределенность и эквивалентность квадратичных форм исследуются на основе теорем Сильвестра. Положительная определенность требует строгой положительности всех главных угловых миноров, а отрицательная требует их чередования, начиная с отрицательного. По закону инерции две квадратичные формы эквивалентны над полем вещественных чисел тогда и только тогда, когда совпадают их сигнатуры (количества положительных и отрицательных собственных значений).

```
In [18]: # Задание 1.18
lam_val = sp.Symbol('lambda', real=True)
Q18 = sp.Matrix([
    [1, lam_val, 5],
    [lam_val, 4, 3],
    [5, 3, 1]
])
d1 = Q18[:1, :1].det()
d2 = Q18[:2, :2].det()
d3 = Q18.det()
print("Delta 1 =", d1)
print("Delta 2 =", d2)
print("Delta 3 =", sp.factor(d3))
print("Delta 3 всегда отрицательна, форма не может быть положительно определенной.")
```

Delta 1 = 1
Delta 2 = 4 - lambda**2
Delta 3 = -lambda**2 + 30*lambda - 105
Delta 3 всегда отрицательна, форма не может быть положительно определенной.

```
In [19]: # Задание 1.19
Q19 = sp.Matrix([
    [lam_val, 1, -1],
    [1, -2, 1],
    [-1, 1, -3]
])
delta1 = Q19[:1, :1].det()
delta2 = Q19[:2, :2].det()
delta3 = Q19.det()
print("Delta 1 =", delta1)
print("Delta 2 =", delta2)
print("Delta 3 =", delta3)
sol19 = sp.solve([delta1 < 0, delta2 > 0, delta3 < 0], lam_val)
print(f"Ответ: {sol19}")
```

Delta 1 = lambda
Delta 2 = -2*lambda - 1
Delta 3 = 5*lambda + 3
Ответ: lambda < -3/5

```
In [20]: # Задание 1.20
Q1_mat = sp.Matrix([
    [2, 4, -2],
    [4, 9, -5],
    [-2, -5, 3]
])
Q2_mat = sp.Matrix([
    [2, -2, -2],
```

```
    [-2, 3, 4],  
    [-2, 4, 6]  
])  
signs_Q1 = [sp.sign(v) for v in Q1_mat.eigenvals().keys()]  
signs_Q2 = [sp.sign(v) for v in Q2_mat.eigenvals().keys()]  
print("Эквивалентны:", sorted(signs_Q1) == sorted(signs_Q2))
```

Эквивалентны: True